

17. Illustrate the deadlock avoidance concept by simulating Banker's algorithm using C.

AIM :

To illustrate the deadlock avoidance concept by simulating Banker's algorithm using C.

CODE:

```
#include <stdio.h>

#define MAX_PROCESSES 5

#define MAX_RESOURCES 3

int available[MAX_RESOURCES] = {3, 3, 2};

int maximum[MAX_PROCESSES][MAX_RESOURCES] = {

    {7, 5, 3},

    {3, 2, 2},

    {9, 0, 2},

    {2, 2, 2},

    {4, 3, 3}

};

int allocation[MAX_PROCESSES][MAX_RESOURCES] = {

    {0, 1, 0},

    {2, 0, 0},

    {3, 0, 2},

    {2, 1, 1},

    {0, 0, 2}

};

int need[MAX_PROCESSES][MAX_RESOURCES];

void calculateNeed()

{

    int i, j;

    for (i = 0; i < MAX_PROCESSES; i++)

    {

        for (j = 0; j < MAX_RESOURCES; j++)
```

```

    {
        need[i][j] = maximum[i][j] - allocation[i][j];
    }
}

int isSafe()
{
    int work[MAX_RESOURCES];
    int finish[MAX_PROCESSES] = {0};
    int safeSequence[MAX_PROCESSES];
    int count = 0, i, j;
    for (i = 0; i < MAX_RESOURCES; i++)
        work[i] = available[i];
    while (count < MAX_PROCESSES)
    {
        int found = 0;
        for (i = 0; i < MAX_PROCESSES; i++)
        {
            if (finish[i] == 0)
            {
                for (j = 0; j < MAX_RESOURCES; j++)
                {
                    if (need[i][j] > work[j])
                        break;
                }
                if (j == MAX_RESOURCES)
                {
                    for (int k = 0; k < MAX_RESOURCES; k++)
                        work[k] += allocation[i][k];

                    safeSequence[count++] = i;
                }
            }
        }
    }
}

```

```

        finish[i] = 1;

        found = 1;
    }
}
}
if (found == 0)
    return 0;
}

printf("\nSafe Sequence: ");
for (i = 0; i < MAX_PROCESSES; i++)
    printf("P%d ", safeSequence[i]);
printf("\n")
return 1;
}

int requestResources(int process, int request[])
{
    int i;
    for (i = 0; i < MAX_RESOURCES; i++)
    {
        if (request[i] > need[process][i])
        {
            printf("Error: Request exceeds process need.\n");
            return 0;
        }
        if (request[i] > available[i])
        {
            printf("Resources not available.\n");
            return 0;
        }
    }
}

for (i = 0; i < MAX_RESOURCES; i++)

```

```

{
    available[i] -= request[i];

    allocation[process][i] += request[i];

    need[process][i] -= request[i];
}

if (isSafe())
{
    return 1;
}
else
{
    for (i = 0; i < MAX_RESOURCES; i++)
    {
        available[i] += request[i];

        allocation[process][i] -= request[i];

        need[process][i] += request[i];
    }

    return 0;
}
}

```

```

int main()
{
    int process;

    int request[MAX_RESOURCES];

    calculateNeed();

    printf("Enter Process Number (0-4): ");

    scanf("%d", &process);

    printf("Enter Request for 3 Resources: ");

    scanf("%d%d%d", &request[0], &request[1], &request[2]);
}

```

```
if (requestResources(process, request))  
    printf("Request Granted.\n");  
else  
    printf("Request Denied.\n");  
return 0;  
}
```

OUTPUT:

```
1  
1 0 2
```

Output

Status : Successfully executed

Time:

0.0000 secs

Memory:

1.808 Mb

Sample Input

```
1  
1 0 2
```

Your Output

```
Enter Process Number (0-4): Enter Request for 3 Resources:  
Safe Sequence: P1 P3 P4 P0 P2  
Request Granted.
```